



Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

A05 - Algoritmo Genético Problema de la Mochila (Knapsack)

Algoritmos Metaheurísticos

Alumno: José Luis Chávez Gómez

5 de octubre de 2026

1. Introducción a los Algoritmos Genéticos

Los algoritmos genéticos (AG) son métodos de optimización inspirados en la evolución biológica por selección natural, propuestos formalmente por Holland [1] y popularizados como herramienta general de búsqueda por Goldberg [2]. A diferencia de los métodos de búsqueda directa revisados en actividades anteriores (búsqueda exhaustiva, búsqueda aleatoria, gradiente descendente y Hill Climbing), que operan sobre una única solución candidata, un AG mantiene una **población** de soluciones que evoluciona a lo largo de **generaciones** sucesivas, de modo que las soluciones de mejor calidad tienen mayor probabilidad de transmitir sus características a la siguiente generación [3, 4].

Cada solución se codifica como un **cromosoma** compuesto de **genes**, y su calidad se mide mediante una función de **aptitud** (*fitness*). El ciclo básico de un AG consta de: (1) **inicialización** de una población aleatoria, (2) **evaluación** de la aptitud de cada individuo, (3) **selección** de padres favoreciendo a los de mejor aptitud, (4) **reproducción** mediante *cruzamiento* (combinación del material genético de dos padres) y *mutación* (alteración aleatoria de genes), (5) **reemplazo** de la población, en ocasiones con *elitismo* para conservar intacto al mejor individuo, y (6) repetición hasta cumplir un criterio de paro.

A diferencia de la actividad anterior (A04), donde el problema del vendedor viajero exigía una representación por **permutaciones** y operadores especiales que preservaran su validez, el problema de la mochila que se aborda en este reporte admite la representación clásica de Holland: un cromosoma **binario**, donde cada gen indica si un objeto se incluye (1) o no (0) en la solución. Esto permite usar los operadores genéticos más estándar de la literatura: cruzamiento de uno o varios puntos y mutación por inversión de bits (*bit-flip*) [2, 4]. El reto particular de esta representación es que el cruzamiento y la mutación pueden producir cromosomas que violen la restricción de capacidad, por lo que se requiere algún mecanismo —penalización o reparación— para mantener la búsqueda dentro de la región factible, como se detalla en la Sección 3.

2. El Problema de la Mochila

El problema de la mochila 0/1 (*0/1 Knapsack Problem*) consiste en seleccionar, de un conjunto de n objetos, cada uno con un peso w_i y un valor v_i , el subconjunto que maximiza el valor total sin exceder una capacidad máxima C . Formalmente, si $x_i \in \{0, 1\}$ indica si el objeto i se incluye, el problema es

$$\max \sum_{i=1}^n v_i x_i \quad \text{sujeto a} \quad \sum_{i=1}^n w_i x_i \leq C. \quad (1)$$

A diferencia de su variante fraccionaria, que se resuelve de forma óptima y eficiente ordenando los objetos por su razón valor/peso, la versión 0/1 —donde cada objeto se toma por completo o se descarta— es un problema de optimización combinatoria clasificado como NP-difícil [5]: para n objetos existen 2^n subconjuntos posibles, por lo que una búsqueda exhaustiva resulta impráctica para instancias grandes (con los $n = 100$ objetos de este reporte, $2^{100} \approx 1,27 \times 10^{30}$ subconjuntos). El problema aparece de forma recurrente en la práctica bajo distintos nombres —asignación de presupuesto, corte de material, carga de contenedores, selección de proyectos de inversión— y es, junto con el vendedor viajero, uno de los problemas de referencia clásicos para evaluar metaheurísticos de optimización combinatoria [6, 5].

3. Algoritmo Genético Implementado

3.1. Datos del problema

Se utilizó el conjunto de 100 objetos proporcionado (`mochila.csv`), cada uno con un peso y un precio generados aleatoriamente ($w_i \in [10, 40]$, $v_i \in [20, 120]$ aproximadamente). La actividad no especificó una capacidad máxima, por lo que se fijó como el **50 % del peso total** de los 100 objetos ($C \approx 1235,66$). Este valor obliga a descartar aproximadamente la mitad del valor disponible, evitando el caso trivial en que la capacidad permite cargar todos los objetos.

3.2. Representación y función objetivo

Cada cromosoma es un vector binario de longitud 100, donde el gen i vale 1 si el objeto i se incluye en la mochila. La aptitud de un cromosoma **factible** es la suma de los precios de los objetos incluidos, ecuación (1), y se busca **maximizarla**.

3.3. Manejo de la restricción: reparación

El cruzamiento y la mutación pueden generar cromosomas cuyo peso total exceda la capacidad. En lugar de penalizar su aptitud (lo que permitiría a individuos no factibles permanecer en la población), se optó por un operador de **reparación**: si un cromosoma excede la capacidad, se retiran uno a uno los objetos incluidos con **peor razón precio/peso**, hasta que el peso vuelva a ser válido. Esta estrategia, común en la literatura de AG para mochila [6], tiene la ventaja de que toda la población permanece siempre factible y de que, al eliminar primero los objetos menos eficientes, suele conservar una solución de buena calidad en lugar de descartarla por completo.

3.4. Selección, cruzamiento y mutación

- **Selección por torneo**: se seleccionan al azar k individuos de la población y se elige al de mayor valor. El tamaño del torneo es un porcentaje fijo de la población (5 %), que regula la presión selectiva [7].
- **Cruzamiento de un punto**: con probabilidad $p_c = 0,9$, dos padres intercambian los genes a partir de una posición de corte elegida al azar, generando dos hijos.
- **Mutación bit-flip**: cada gen de un hijo tiene una probabilidad $p_m = 1/n$ de invertirse (de 0 a 1 o viceversa), el valor estándar recomendado para cromosomas binarios de longitud n [2].
- **Reparación**: se aplica después de la mutación, garantizando que todos los individuos de la nueva generación sean factibles.
- **Elitismo**: el mejor cromosoma de cada generación pasa sin modificarse a la siguiente, de modo que el mejor valor encontrado nunca empeora.

3.5. Parámetros utilizados

Parámetro	Valor
Número de objetos	100
Capacidad de la mochila	$\approx 1235,66$ (50 % del peso total)
Tamaño de la población	150
Número de generaciones	300
Tamaño del torneo	7 (5 % de la población)
Probabilidad de cruzamiento	0.9
Probabilidad de mutación por gen	1/100
Elitismo	1 individuo
Semilla aleatoria	42

Cuadro 1: Parámetros del algoritmo genético para el problema de la mochila.

4. Código Fuente

```
1  """ Bibliotecas necesarias
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  import random
6
7  """ Carga de datos
8  datos = pd.read_csv("mochila.csv")
9  PESOS = datos["peso"].values
10 PRECIOS = datos["precio"].values
11 NUM_ITEMS = len(PESOS)
12
13 # Capacidad de la mochila: no fue especificada en la tarea, así que se fija
14 # como el 50% del peso total de todos los objetos. Esto obliga a descartar
15 # aproximadamente la mitad del valor disponible, lo cual hace el problema
16 # no trivial (si la capacidad fuera mayor a la suma de todos los pesos,
17 # la solución óptima sería simplemente llevar todo).
18 CAPACIDAD = 0.5 * PESOS.sum()
19
20 """ Parámetros del algoritmo genético
21 POP_SIZE = 150
22 GENERATIONS = 300
23 TOURNAMENT_PERCENT = 0.05
24 TOURNAMENT_SIZE = max(2, int(POP_SIZE * TOURNAMENT_PERCENT))
25 PROB_CRUZA = 0.9
26 PROB_MUTACION = 1.0 / NUM_ITEMS # en promedio, un gen mutado por cromosoma
27 ELITISM = True
28 SEED = 42
29
30 if SEED is not None:
31     random.seed(SEED)
32     np.random.seed(SEED)
33
34 """ Función objetivo (con reparación de soluciones no válidas)
35 def reparar(cromosoma):
36     """
37     Si el peso total excede la capacidad, se van quitando objetos hasta que
38     el cromosoma sea válido. Se eliminan primero los objetos con peor razón
39     precio/peso (los "menos eficientes"), ya que son los que menos aportan
40     por unidad de peso que ocupan. Esto evita descartar soluciones completas
41     por una pequeña violación de la restricción (a diferencia de una
```

```

42 penalización en la función de costo) y mantiene siempre cromosomas
43 factibles en la población.
44 """
45 peso_total = np.dot(cromosoma, PESOS)
46 if peso_total <= CAPACIDAD:
47     return cromosoma
48
49 cromosoma = cromosoma.copy()
50 indices_incluidos = [i for i in range(NUM_ITEMS) if cromosoma[i] == 1]
51 # Ordenar los objetos incluidos de peor a mejor razón precio/peso
52 indices_incluidos.sort(key=lambda i: PRECIOS[i] / PESOS[i])
53
54 for i in indices_incluidos:
55     if peso_total <= CAPACIDAD:
56         break
57     cromosoma[i] = 0
58     peso_total -= PESOS[i]
59
60 return cromosoma
61
62 def evaluar(cromosoma):
63     """Valor total de los objetos incluidos (el cromosoma ya debe ser válido)."""
64     return float(np.dot(cromosoma, PRECIOS))
65
66 %% Funciones de ayuda del algoritmo genético binario
67 def crear_cromosoma():
68     """Cromosoma binario: 1 si el objeto se incluye, 0 si no."""
69     cromosoma = np.random.randint(0, 2, NUM_ITEMS)
70     return reparar(cromosoma)
71
72 def crear_poblacion():
73     return [crear_cromosoma() for _ in range(POP_SIZE)]
74
75 def torneo(poblacion, valores):
76     """Selección por torneo: gana el cromosoma de mayor valor."""
77     participantes_idx = random.sample(range(POP_SIZE), TOURNAMENT_SIZE)
78     valores_participantes = [valores[i] for i in participantes_idx]
79     return poblacion[participantes_idx[int(np.argmax(valores_participantes))]]
80
81 %% Operadores genéticos
82 def cruza_un_punto(padre1, padre2):
83     """Cruzamiento de un punto: intercambia los genes a partir de un corte
84     aleatorio."""
85     if random.random() > PROB_CRUZA:
86         return padre1.copy(), padre2.copy()
87     punto = random.randint(1, NUM_ITEMS - 1)
88     hijo1 = np.concatenate([padre1[:punto], padre2[punto:]]
89                             hijo2 = np.concatenate([padre2[:punto], padre1[punto:]]
90     return hijo1, hijo2
91
92 def mutacion(cromosoma):
93     """Mutación bit-flip: cada gen tiene una probabilidad PROB_MUTACION de
94     invertirse."""
95     cromosoma = cromosoma.copy()
96     for i in range(NUM_ITEMS):
97         if random.random() < PROB_MUTACION:
98             cromosoma[i] = 1 - cromosoma[i]
99     return cromosoma
100
101 %% Algoritmo genético
102 def algoritmo_genetico():
103     """

```

```

102 Regresa:
103     mejor_cromosoma_global : mejor solución encontrada en todas las
                             generaciones
104     mejor_valor_global     : su valor total
105     hist_mejor             : mejor valor de cada generación
106     hist_promedio          : valor promedio de cada generación
107     """
108     poblacion = crear_poblacion()
109
110     hist_mejor = []
111     hist_promedio = []
112     mejor_cromosoma_global = None
113     mejor_valor_global = -np.inf
114
115     for g in range(GENERATIONS):
116         # 1. Evaluar toda la población
117         valores = [evaluar(c) for c in poblacion]
118
119         # 2. Mejor de la generación
120         mejor_idx = int(np.argmax(valores))
121         mejor_cromosoma = poblacion[mejor_idx].copy()
122         mejor_valor = valores[mejor_idx]
123
124         hist_mejor.append(mejor_valor)
125         hist_promedio.append(float(np.mean(valores)))
126
127         # 3. Actualizar el mejor global
128         if mejor_valor > mejor_valor_global:
129             mejor_valor_global = mejor_valor
130             mejor_cromosoma_global = mejor_cromosoma.copy()
131
132         # 4. Generar hijos: selección por torneo + cruza + mutación
133         hijos = []
134         while len(hijos) < POP_SIZE:
135             padre1 = torneo(poblacion, valores)
136             padre2 = torneo(poblacion, valores)
137
138             hijo1, hijo2 = cruza_un_punto(padre1, padre2)
139             hijo1 = reparar(mutacion(hijo1))
140             hijo2 = reparar(mutacion(hijo2))
141
142             hijos.append(hijo1)
143             if len(hijos) < POP_SIZE:
144                 hijos.append(hijo2)
145
146         # 5. Elitismo: el mejor de la generación sobrevive sin modificarse
147         if ELITISM:
148             hijos[0] = mejor_cromosoma.copy()
149
150         poblacion = hijos
151
152         if g % 50 == 0:
153             print(f"Generacion {g}: Mejor valor = {mejor_valor:.2f} "
154                   f"Peso usado = {np.dot(mejor_cromosoma, PESOS):.2f} / {
155                       CAPACIDAD:.2f}")
156
157     return mejor_cromosoma_global, mejor_valor_global, hist_mejor, hist_promedio
158
159 %% Gráficas
160 def graficar_historial(hist_mejor, hist_promedio, nombre_archivo):
161     plt.figure(figsize=(8, 5))
162     plt.plot(hist_promedio, label='Promedio de la población', color='tab:orange',
163             , alpha=0.8)

```

```

162     plt.plot(hist_mejor, label='Mejor de la generación', color='tab:blue',
163             linewidth=2)
164     plt.xlabel('Generación')
165     plt.ylabel('Valor total de la mochila')
166     plt.title('Historial de la función de costo')
167     plt.legend()
168     plt.grid(alpha=0.3)
169     plt.savefig(nombre_archivo, dpi=200, bbox_inches='tight')
170
171 def graficar_solucion(cromosoma, nombre_archivo):
172     """
173     Dispersión peso vs. precio de los 100 objetos, distinguiendo los
174     incluidos en la mochila de los descartados.
175     """
176     incluidos = cromosoma == 1
177     plt.figure(figsize=(7, 6))
178     plt.scatter(PESOS[~incluidos], PRECIOS[~incluidos], color='lightgray',
179               label='Descartado', alpha=0.8)
180     plt.scatter(PESOS[incluidos], PRECIOS[incluidos], color='tab:green',
181               label='Incluido', alpha=0.9)
182     plt.xlabel('Peso')
183     plt.ylabel('Precio')
184     plt.title('Objetos incluidos en la mejor solución encontrada')
185     plt.legend()
186     plt.grid(alpha=0.3)
187     plt.savefig(nombre_archivo, dpi=200, bbox_inches='tight')
188
189 %% Programa principal
190 if __name__ == "__main__":
191     mejor_cromosoma, mejor_valor, hist_mejor, hist_promedio = algoritmo_genetico
192     ()
193
194     peso_usado = np.dot(mejor_cromosoma, PESOS)
195     num_incluidos = int(mejor_cromosoma.sum())
196
197     print("\nMejor valor encontrado:", mejor_valor)
198     print(f"Peso utilizado: {peso_usado:.2f} / {CAPACIDAD:.2f}")
199     print(f"Objetos incluidos: {num_incluidos} de {NUM_ITEMS}")
200     print(f"Mejor valor de la población inicial: {hist_mejor[0]:.2f}")
201
202     graficar_historial(hist_mejor, hist_promedio, "historial_costo_mochila.png")
203     graficar_solucion(mejor_cromosoma, "solucion_mochila.png")
204     plt.show()

```

5. Resultados

Con la semilla 42, el mejor valor de la población inicial fue 4492,88; el algoritmo lo incrementó hasta 5386,08, una mejora de aproximadamente 19,9%, utilizando un peso de 1233,85 de los 1235,66 disponibles (99.85 % de la capacidad) e incluyendo 58 de los 100 objetos. Como referencia de calidad, la relajación fraccionaria del problema (permitiendo tomar fracciones de objetos, resoluble de forma óptima y eficiente) da una cota superior de 5396,44; la solución entera encontrada por el algoritmo genético alcanza el 99,8 % de esa cota, lo que indica una solución muy cercana al óptimo real del problema 0/1.

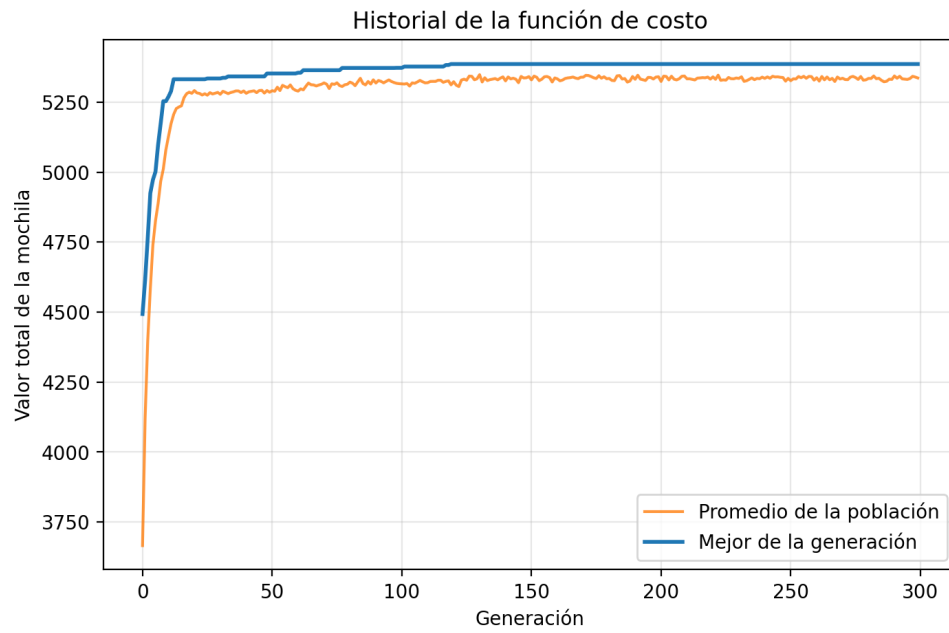


Figura 1: Historial de la función de costo: mejor valor de cada generación y valor promedio de la población.

La Figura 1 muestra que la mejor solución mejora rápidamente durante las primeras 20 generaciones y se estabiliza alrededor de la generación 120, gracias al elitismo. El valor promedio de la población se mantiene por debajo del mejor individuo durante toda la corrida, reflejo de que la mutación introduce continuamente diversidad (y, con ella, individuos de menor calidad) en cada generación.

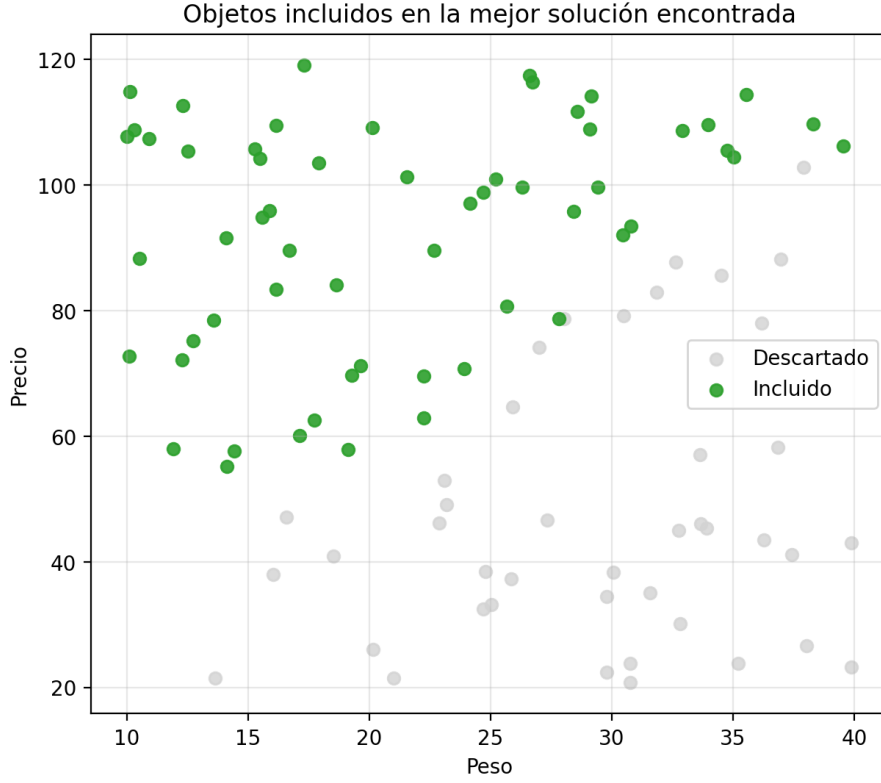


Figura 2: Objetos incluidos (verde) y descartados (gris) en la mejor solución encontrada, en función de su peso y precio.

La Figura 2 confirma el comportamiento esperado: los objetos incluidos se concentran en la región de mayor precio para un peso dado (mejor razón precio/peso), mientras que los descartados se concentran en la región de menor precio relativo, consistente con la intuición de la relajación fraccionaria del problema.

Métrica	Valor
Mejor valor de la población inicial	4492,88
Mejor valor encontrado	5386,08
Mejora respecto a la población inicial	19,9 %
Cota superior (relajación fraccionaria)	5396,44
Calidad respecto a la cota superior	99,8 %
Peso utilizado	1233,85 / 1235,66
Objetos incluidos	58 de 100

Cuadro 2: Resumen de resultados de la corrida con semilla 42.

6. Conclusión

El algoritmo genético binario resolvió de forma efectiva una instancia del problema de la mochila 0/1 con 100 objetos, un problema cuyo espacio de búsqueda contiene del orden de 10^{30} subconjuntos y que, por tanto, está muy fuera del alcance de la búsqueda exhaustiva de la actividad A03. Con una representación binaria natural para el problema, el operador de reparación basado en la razón precio/peso resultó clave: permitió que el cruzamiento y la mutación exploraran libremente el espacio de soluciones sin necesidad de penalizar ni descartar cromosomas no factibles, manteniendo siempre una población válida. El resultado final, a un 99,8 % de la cota

superior dada por la relajación fraccionaria, sugiere que la solución encontrada está muy cerca del óptimo entero real.

Comparado con el algoritmo genético ordinal de la actividad A04, este problema ilustra cómo la elección de la representación del cromosoma y de los operadores genéticos depende directamente de la estructura del problema: mientras que el vendedor viajero exige preservar el orden mediante operadores especiales (reversa, enroque), la mochila admite la representación binaria clásica con cruzamiento y mutación estándar, a cambio de requerir un mecanismo adicional —en este caso, reparación— para manejar su restricción de capacidad. Ambos casos refuerzan una idea central de los algoritmos genéticos: su flexibilidad para adaptarse a distintas estructuras de problema mediante el diseño adecuado de la codificación y los operadores, manteniendo el mismo esquema general de selección, reproducción y reemplazo poblacional.

Referencias

- [1] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [2] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- [3] Simon, D. (2013). *Evolutionary Optimization Algorithms: Biologically-Inspired and Population-Based Approaches to Computer Intelligence*. John Wiley & Sons.
- [4] Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer.
- [5] Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [6] Martello, S., & Toth, P. (1990). *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons.
- [7] Miller, B. L., & Goldberg, D. E. (1995). Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, 9(3), 193–212.